

BÁO CÁO TOÀN DIỆN DỰ ÁN QUẢN LÝ KHÁCH SẠN NOSQL

KIẾN TRÚC HỆ THỐNG — PHẦN VIỆC NGUYỄN ANH QUÂN (QN) TỪ DB ĐẾN UI VÀ TƯ DUY THIẾT KẾ TRUY VẤN CƠ SỞ DỮ LIỆU CASSANDRA / ASTRADB

Sinh viên phụ trách: Nguyễn Anh Quân (QN)	Học phần: Cơ sở dữ liệu NoSQL — HUIT
Phạm vi phụ trách: Đặt phòng, Giao dịch BATCH & Hóa đơn	Trọng số Sprint: 13 SP (~37.5% toàn hệ thống)
Công nghệ: AstraDB Cloud, Python Flask, Tailwind v4	Kết quả Test: 94/94 Unit Tests PASS · 79/79 E2E PASS

1. TỔNG QUAN HỆ THỐNG VÀ CÁC LUỒNG VẬN HÀNH (WORKFLOWS)

1.1. Kiến trúc phân tầng 3 lớp (3-Tier Distributed Architecture):

Hệ thống được thiết kế theo kiến trúc chuẩn Web Doanh nghiệp kết hợp Cơ sở dữ liệu phân tán NoSQL:

- Tầng Giao diện (Presentation Layer): Trình duyệt web render HTML5 qua Jinja2 Templates, giao diện hiện đại xây dựng trên Tailwind CSS v4, tối ưu hiển thị dạng Canvas rộng (Full-width), tích hợp biểu đồ Chart.js trực quan.
- Tầng Điều khiển & Dịch vụ (Controller & Service Layer): Xây dựng trên Python Flask chia thành 3 Blueprint độc lập (hotel_bp, booking_bp, dashboard_bp). Tầng Service đóng gói toàn bộ Prepared Statements, cơ chế Cache In-Memory TTL 30s và xử lý tính toán số đêm, doanh thu, thanh toán cọc.
- Tầng Cơ sở dữ liệu phân tán (Distributed Database Layer): Cụm Cassandra trên nền tảng DataStax AstraDB Cloud, kết nối bảo mật qua Secure Connect Bundle Singleton (get_session()). Gồm 6 bảng phục vụ chính xác 5 câu hỏi nghiệp vụ (Query Q1 đến Q5).

1.2. Ba luồng nghiệp vụ cốt lõi xuyên suốt hệ thống:

- Luồng Đặt phòng mới (Booking Flow): Người dùng tìm kiếm phòng trống theo bộ lọc đa tiêu chí tại /bookings -> Bấm đặt phòng -> Modal tự tính tiền theo số đêm và chọn cọc (100%, 50%, 0%) -> Hệ thống chạy Query Q5 (Atomic BATCH) ghi đồng thời vào bookings_by_guest và bookings_by_hotel_date -> Tự động sinh hóa đơn invoices_by_booking (Q4) -> Chuyển trạng thái phòng sang OCCUPIED, lưu tên khách và mã booking -> Xóa cache -> Điều hướng đến Hóa đơn để in ấn.
- Luồng Thu tiền còn lại (Settlement Flow): Lọc hóa đơn có trạng thái PARTIAL tại /invoices -> Bấm 'Thu tiền còn lại' -> Hệ thống cập nhật bảng hóa đơn: payment_status = 'PAID', deposit_amount = total_amount, remaining_amount = 0 -> Cập nhật phương thức thanh toán -> Invalidate cache.
- Luồng Trả phòng & Trả sớm (Check-out Flow): Lễ tân bấm 'Trả phòng' tại trang chi tiết phòng -> Hệ thống kiểm tra nếu khách trả sớm hơn dự kiến thì ghi lại ngày trả thực tế vào check_out_date ở bảng khách để thống kê số đêm chuẩn (không hoàn tiền, hóa đơn giữ nguyên) -> BATCH UPDATE chuyển trạng thái booking sang COMPLETED ở cả 2 bảng -> Giải phóng phòng về AVAILABLE, xóa tên khách cũ về NULL.

2. CHI TIẾT PHẦN VIỆC CỦA NGUYỄN ANH QUÂN (QN) TỪ DB ĐẾN UI

Tầng kiến trúc	Nhiệm vụ cụ thể (Tasks)	Mã nguồn phụ trách	Kết quả đạt được
Database (CQL Schema)	- Tạo bảng bookings_by_guest (Q2) - Tạo bảng bookings_by_hotel_date (Q3) - Tạo bảng invoices_by_booking (Q4) - Mở rộng cột cọc: deposit_amount, remaining_amount	cql/schema.cql (Dòng 74–111)	Mô hình dữ liệu phi chuẩn hóa tối ưu, khóa chính chuẩn xác, không quét toàn bảng.
Service Layer (Business Logic)	- Cài đặt BATCH INSERT Q5 - Truy vấn lịch sử khách Q2 - Truy vấn lễ tân theo ngày Q3 - Tra cứu & tạo hóa đơn Q4 - Thu nốt tiền cọc (Settle payment) - BATCH UPDATE trả phòng sớm - In-Memory Caching TTL 30s	services/booking_service.py (1.320 dòng code)	Ghi đồng thời nguyên tử, giải quyết bài toán tính nhất quán dữ liệu phân tán, xử lý ngoại lệ và fallback an toàn.

Controller (Flask Routes)	• 13 Route Endpoints (Web forms & JSON APIs) • API phòng trống theo khách sạn • API chi tiết phòng & booking đang ở • Xử lý Validation ràng buộc ngày đi > ngày đến, chặn trùng phòng bận	routes/ booking_routes.py (860 dòng code)	Điều hướng mượt mà, phân tách rõ ràng giữa Web View và RESTful API cho modal.
UI / Frontend (Templates)	• bookings.html: Bộ lọc phòng 5 tiêu chí, Modal đặt phòng, Bảng lịch sử đặt phòng • invoices.html: Phiếu hóa đơn cao cấp nhận diện Mường Thanh, Bảng quản lý hóa đơn, Modal thu tiền cọc	templates/ bookings.html invoices.html	Giao diện sắc nét, chuẩn responsive, có nút In hóa đơn trực tiếp, trạng thái màu sắc rõ ràng.

3. BẢNG KÊ CỨU TOÀN BỘ CÁC TRUY VẤN CQL VỀ ĐẶT PHÒNG & HÓA ĐƠN

QUERY 1 & 2: DDL Khởi tạo bảng dữ liệu Đặt phòng (bookings_by_guest & bookings_by_hotel_date)

```
-- Bảng 4: Lịch sử theo khách (Query Q2: PK = guest_id, CK = booking_id)
CREATE TABLE IF NOT EXISTS bookings_by_guest (
  guest_id text, booking_id text, hotel_id text, room_number text,
  check_in_date date, check_out_date date, status text, total_amount decimal,
  PRIMARY KEY (guest_id, booking_id)
);

-- Bảng 5: Lịch sử theo khách sạn và ngày (Query Q3: Composite PK = ((hotel_id, check_in_date)), CK = booking_id)
CREATE TABLE IF NOT EXISTS bookings_by_hotel_date (
  hotel_id text, check_in_date date, booking_id text,
  guest_id text, guest_name text, room_number text, status text,
  PRIMARY KEY ((hotel_id, check_in_date), booking_id)
);
```

QUERY 3: DDL Khởi tạo bảng Hóa đơn thanh toán (invoices_by_booking - Query Q4)

```
-- Bảng 6: Hóa đơn theo đơn đặt phòng (PK = booking_id, CK = invoice_id)
CREATE TABLE IF NOT EXISTS invoices_by_booking (
  booking_id text, invoice_id text, guest_name text, hotel_id text,
  issue_date date, payment_method text, payment_status text, total_amount decimal,
  deposit_amount decimal, remaining_amount decimal,
  PRIMARY KEY (booking_id, invoice_id)
);
```

QUERY 4: [TRỌNG TÂM ĐỒ ÁN] Atomic BATCH INSERT Đặt phòng (Query Q5 Đề Cương)

```
BEGIN BATCH
  INSERT INTO bookings_by_guest (guest_id, booking_id, hotel_id, room_number,
    check_in_date, check_out_date, status, total_amount)
  VALUES (?, ?, ?, ?, ?, ?, ?, ?);

  INSERT INTO bookings_by_hotel_date (hotel_id, check_in_date, booking_id,
    guest_id, guest_name, room_number, status)
  VALUES (?, ?, ?, ?, ?, ?, ?);
APPLY BATCH;
```

Vị trí cài đặt: services/booking_service.py hàm create_booking_batch(...) (dòng 80–110). Dùng BatchStatement() của Cassandra driver để đảm bảo dữ liệu ghi đồng thời vào cả 2 bảng phân tán.

QUERY 5: Tra cứu lịch sử đặt phòng theo khách hàng (Query Q2 Đề Cương)

```
SELECT guest_id, booking_id, hotel_id, room_number,
  check_in_date, check_out_date, status, total_amount
FROM bookings_by_guest
WHERE guest_id = ?;
```

Vị trí cài đặt: services/booking_service.py hàm get_bookings_by_guest(guest_id) (dòng 256–290). Truy vấn O(1) theo Partition Key guest_id, không quét toàn bảng.

QUERY 6: Tra cứu khách check-in theo khách sạn và ngày (Query Q3 Đề Cương)

```
SELECT hotel_id, check_in_date, booking_id,
       guest_id, guest_name, room_number, status
FROM bookings_by_hotel_date
WHERE hotel_id = ? AND check_in_date = ?;
```

Vị trí cài đặt: services/booking_service.py hàm get_bookings_by_hotel_date(hotel_id, check_in_date) (dòng 296–343). Truy vấn O(1) theo Composite Partition Key (hotel_id, check_in_date).

QUERY 7 & 8: Tạo mới và Tra cứu chi tiết hóa đơn thanh toán (Query Q4 Đề Cương)

```
-- 1. Tạo hóa đơn kèm quản lý đặt cọc
INSERT INTO invoices_by_booking (
    booking_id, invoice_id, guest_name, hotel_id,
    issue_date, payment_method, payment_status, total_amount,
    deposit_amount, remaining_amount
) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
```

```
-- 2. Tra cứu chi tiết hóa đơn theo mã đặt phòng
SELECT booking_id, invoice_id, guest_name, hotel_id,
       issue_date, payment_method, payment_status, total_amount,
       deposit_amount, remaining_amount
FROM invoices_by_booking
WHERE booking_id = ?;
```

Vị trí cài đặt: create_invoice(...) (dòng 406–560) và get_invoice_by_booking(booking_id) (dòng 349–404).

QUERY 9: Cập nhật thanh toán nốt tiền cọc (Settle Payment)

```
UPDATE invoices_by_booking
SET payment_status = ?, deposit_amount = ?, remaining_amount = ?, payment_method = ?
WHERE booking_id = ? AND invoice_id = ?;
```

Vị trí cài đặt: settle_invoice_payment(...) (dòng 1071–1186). Cập nhật sang PAID khi khách đóng nốt số tiền còn lại.

QUERY 10: Đóng booking khi khách trả phòng bằng BATCH UPDATE (Trả phòng sớm)

```
BEGIN BATCH
-- Cập nhật bảng khách (ghi nhận ngày trả thực tế nếu trả sớm)
UPDATE bookings_by_guest SET status = 'COMPLETED', check_out_date = ?
WHERE guest_id = ? AND booking_id = ?;

-- Cập nhật bảng lễ tân
UPDATE bookings_by_hotel_date SET status = 'COMPLETED'
WHERE hotel_id = ? AND check_in_date = ? AND booking_id = ?;
APPLY BATCH;
```

Vị trí cài đặt: complete_booking(...) (dòng 1194–1320). Đồng bộ trạng thái COMPLETED ở cả 2 bảng, không giảm giá và không hoàn tiền.

QUERY 11: Cập nhật phòng khi Đặt và khi Trả phòng (rooms_by_hotel)

```
-- Khi Đặt phòng thành công:
UPDATE rooms_by_hotel
SET status = 'OCCUPIED', is_available = false, current_guest_name = ?, current_booking_id = ?
WHERE hotel_id = ? AND room_number = ?;

-- Khi Khách trả phòng (Giải phóng phòng trống):
UPDATE rooms_by_hotel
SET status = 'AVAILABLE', is_available = true, current_guest_name = null, current_booking_id = null
WHERE hotel_id = ? AND room_number = ?;
```

4. TƯ DUY NGƯỢC LẠI: TẠI SAO LẠI THIẾT KẾ QUERY & DỮ LIỆU NHƯ VẬY?

Câu hỏi bản chất	Cách làm & Hạn chế của SQL (RDBMS)	Tư duy & Giải pháp trong Cassandra (NoSQL)
------------------	------------------------------------	--

1. Tại sao có 2 bảng Đặt phòng thay vì 1?	Dùng 1 bảng bookings. Khi cần xem lịch sử khách hay lễ tân thì dùng lệnh SELECT ... WHERE ... và tạo Secondary Index.	Cassandra phân tán dữ liệu theo hash Partition Key. Không có JOIN. Nếu dùng 1 bảng mà query không đúng Partition Key sẽ bị Full Cluster Scan làm sập hệ thống. Tách 2 bảng (Denormalization) giúp cả 2 góc nhìn đều đạt tốc độ \$O(1)\$ (< 5ms).
2. Tại sao bắt buộc dùng BATCH STATEMENT?	SQL dùng ACID Transaction (BEGIN TRANSACTION ... COMMIT). Dữ liệu chỉ nằm ở 1 bảng nên không lo lệch pha giữa các bảng trùng lặp.	Vì có 2 bảng lưu cùng 1 sự kiện đặt phòng, nếu chạy 2 lệnh INSERT rồi, rủi ro sập mạng giữa chừng sẽ làm bảng khách có đơn mà bảng lễ tân không có. BATCH đảm bảo tính nguyên tử (Atomicity) trên cụm phân tán.
3. Tại sao bảng lễ tân dùng Composite Partition Key?	SQL chỉ cần đánh chỉ mục (Index) trên cột hotel_id và check_in_date.	Nếu chỉ lấy hotel_id làm Partition Key, qua nhiều năm một khách sạn lớn sẽ gom hàng trăm nghìn dòng vào 1 Partition, vượt mốc 100MB khuyến nghị gây Hotspot. Dùng ((hotel_id, check_in_date)) giúp mỗi ngày là 1 partition nhỏ (~vài chục KB) siêu tốc.
4. Tại sao Denormalize tên khách vào phòng?	Dùng lệnh JOIN giữa bảng rooms, bookings và guests để lấy tên khách đang ở.	Cassandra không có JOIN. Nếu không lưu sẵn tên khách vào dòng phòng đó, mỗi khi xem chi tiết phòng sẽ phải quét toàn bộ bảng đặt phòng. Denormalize current_guest_name giúp xem chi tiết phòng chỉ tốn đúng 1 lần đọc duy nhất!
5. Tại sao Trả phòng sớm không hoàn tiền?	Tùy chính sách, thường tính lại số đêm thực tế và hoàn tiền thừa.	Theo quy chuẩn vận hành chuỗi Mường Thanh đã thống nhất: Khách trả sớm giữ phòng nên không giảm giá/hoàn tiền. Việc lưu ngày trả thực tế chỉ để tính chỉ số công suất phòng (Occupancy Rate/ADR) cho Dashboard.
6. Tại sao cần In-Memory Cache TTL 30s?	SQL Server thường có buffer pool tự động trên RAM máy chủ.	AstraDB là Cloud DB tính phí theo Request Units và có độ trễ internet (network hop). Cache 30s ở RAM máy chủ web giúp người dùng load trang tức thì, giảm 80% truy vấn lặp lại vào cloud.

5. CẨM NANG THUYẾT TRÌNH BẢO VỆ ĐỒ ÁN TRƯỚC HỘI ĐỒNG

Ba câu nói then chốt giúp đạt điểm tối đa khi thầy cô hỏi phân biện:

1. Khi thầy cô hỏi 'Tại sao cơ sở dữ liệu lại có nhiều bảng trùng lặp dữ liệu thế này?':
'Thưa thầy cô, đây là nguyên lý cốt lõi của NoSQL Cassandra: Query-First Design và Denormalization. Trong RDBMS, ta chuẩn hóa 3NF để tiết kiệm ổ đĩa và dùng JOIN. Nhưng Cassandra là hệ CSDL phân tán quy mô lớn, không hỗ trợ JOIN vì JOIN qua mạng giữa các node sẽ gây nghẽn cổ chai. Do đó, ta thiết kế mỗi bảng chuyên biệt cho đúng một truy vấn đọc. Dung lượng ổ đĩa ngày nay rất rẻ, nhưng độ trễ truy vấn O(1) là yếu tố sống còn.'

2. Khi thầy cô hỏi 'BATCH STATEMENT ở đây dùng để làm gì? Có giống Transaction trong SQL không?':
'Dạ thưa thầy cô, BATCH trong Cassandra dùng để đảm bảo tính nguyên tử (Atomicity) khi ghi đồng thời vào nhiều bảng phân tán (ở đây là bookings_by_guest và bookings_by_hotel_date). Cassandra sử dụng cơ chế batchlog trên coordinator node để đảm bảo hoặc cả hai bảng cùng được ghi hoặc nếu gặp sự cố mạng thì các node khác sẽ replay lại log, ngăn chặn hoàn toàn tình trạng mất đồng bộ dữ liệu.'

3. Khi thầy cô hỏi 'Tại sao không dùng ALLOW FILTERING để tìm kiếm cho tiện?':
'Dạ thưa thầy cô, ALLOW FILTERING trong môi trường production của Cassandra là một anti-pattern nghiêm trọng. Nó bắt buộc Cassandra phải quét qua tất cả các node trong toàn cụm, gây ngốn CPU và timeout. Trong toàn bộ code của nhóm em, mọi câu lệnh truy vấn chính đều đánh thẳng vào Partition Key để đạt hiệu năng O(1).'